```java
// Example 01: Printing 1 to 4 using while loop
public static void show1to4While() {
    int i = 1;
    while (i <= 4) {
        System.out.println(i);
        i++;
    }
}
```

```java
public static void show1to4(int i) {
    if (i > 4) {
        return;
    } else {
        System.out.println(i);
        show1to4(i + 1);
    }
}
```

```java
public static void show4to1While() {
    int i = 4;
    while (i > 0) {
        System.out.println(i);
        i--;
    }
}
```

```java
public static void show4to1(int i) {
    if (i == 0) {
        return;
    } else {
        System.out.println(i);
        show4to1(i - 1);
    }
}
```

```java
public static void show4to1ReverseOrder(int i) {
    if (i > 4) {
        return;
    } else {
        show4to1ReverseOrder(i + 1);
        System.out.println(i);
    }
}
```

```java
public static int sum1toNWhile( int n) {
    int sum = 0;
    int i=1;
    while (i <= n) {
        sum += i;
        i++;
    }
    return sum;
}
```

```java
public static int sum1toN(int i, int n) {
    if (i == n) {
        return n;
    } else {
        return i + sum1toN(i + 1, n);
    }
}
```

```java
public static void printArrayForwardWhile(int[] arr) {
    int i = 0;
    while (i < arr.length) {
        System.out.print(arr[i] + " ");
        i++;
    }
    System.out.println();
}
```

```java
public static void arrForwardPrinting(int i, int[] arr) {
    if (i == arr.length) return;
    else {
        System.out.print(arr[i] + " ");
        arrForwardPrinting(i + 1, arr);
    }
}
```

```java
public static void printArrayBackwardWhile(int[] arr) {
    int i = arr.length - 1;
    while (i >= 0) {
        System.out.print(arr[i] + " ");
        i--;
    }
    System.out.println();
}
```

```java
public static void printArrayBackwardRec(int i, int[] arr) {
    if (i < 0) {
        return;
    } else {
        System.out.print(arr[i] + " ");
        printArrayBackwardRec(i - 1, arr);
    }
}
```

```java
public static void arrBkPrintOrderChange(int i, int[] arr) {
    if (i == arr.length) return;
    else {
        arrBkPrintOrderChange(i + 1, arr);
        System.out.print(arr[i] + " ");
    }
}
```

```java
// Example 08: Summation of array elements - ITERATIVE
public static int arrSumIterative(int[] arr) {
    int total = 0;
    int i = 0;
    while (i < arr.length) {
        total += arr[i];
        i++;
    }
    return total;
}
```

```java
public static int arrSum(int i, int[] arr) {
    if (i == arr.length - 1) {
        return arr[i];
    } else {
        return arr[i] + arrSum(i + 1, arr);
    }
}
```

```java
// Example 10: Weird sum: add even, subtract odd (while-loop version)
public static int arrWeirdSumWhile(int[] arr) {
    int i = 0;
    int total = 0;

    while (i < arr.length) {
        total += evenOdd(arr[i]);
        i++;
    }

    return total;
}

// Helper: returns +n for even, -n for odd, and prints the sign
public static int evenOdd(int n) {
    if (n % 2 == 1) {
        return -n;
    } else {
        return n;
    }
}
```

```java
public static int arrWeirdSum(int i, int[] arr) {
    if (i == arr.length - 1) {
        return evenOdd(arr[i]);
    } else {
        return evenOdd(arr[i]) + arrWeirdSum(i + 1, arr);
    }
}

public static int evenOdd(int n) {
    if (n % 2 == 1) {
        System.out.print("-" + n);
        return -n;
    } else {
        System.out.print("+" + n);
        return n;
    }
}
```

```java
public static void printLL_iterative(Node head) {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.elem + " -> ");
        temp = temp.next;
    }
    System.out.println("null");
}
```

```java
public static void printForward_recursive(Node temp) {
    if (temp == null) {
        System.out.println("null");
        return;
    } else {
        System.out.print(temp.elem + " => ");
        printForward_recursive(temp.next);
    }
}
```

```java
public static void printBackward_recursive(Node temp) {
    if (temp == null) {
        return;
    } else {
        printBackward_recursive(temp.next);
        System.out.print(temp.elem + " <= ");
    }
}
```

```java
public static Node arr2LL_iterative(int[] arr) {
    if (arr.length == 0) {
        return null;  // Handle empty array
    }
    Node head = new Node(arr[0]);
    Node current = head;
    int i = 1;
    while (i < arr.length) {
        Node newNode = new Node(arr[i]);
        current.next = newNode;
        current = current.next;
        i++;
    }
    return head;
}
```

```java
// Convert array to linked list (recursive)
public static Node arr2LL_recursive(int[] arr, int index) {
    if (index == arr.length) {
        return null;
    }
    Node current = new Node(arr[index]);
    current.next = arr2LL_recursive(arr, index + 1);
    return current;
}
```